Robot Perception Midterm Project
Team SWIFT

[Ashwik Manoj Tatineni] (at6287)
[Jotheesh Reddy Kummathi] (jrk8067)
[Sai Pranay Reddy Jonnala] (sj4526)
[Sourabh Misal] (srm9726)

Problem Statement

Goal: We have to navigate using the visual input the agent must localize, plan and move intelligently

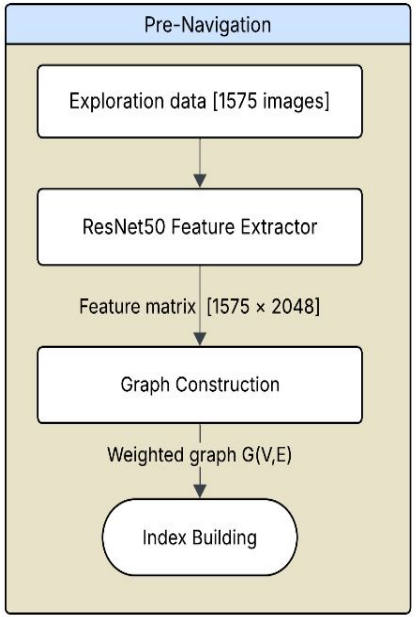
Challenges:

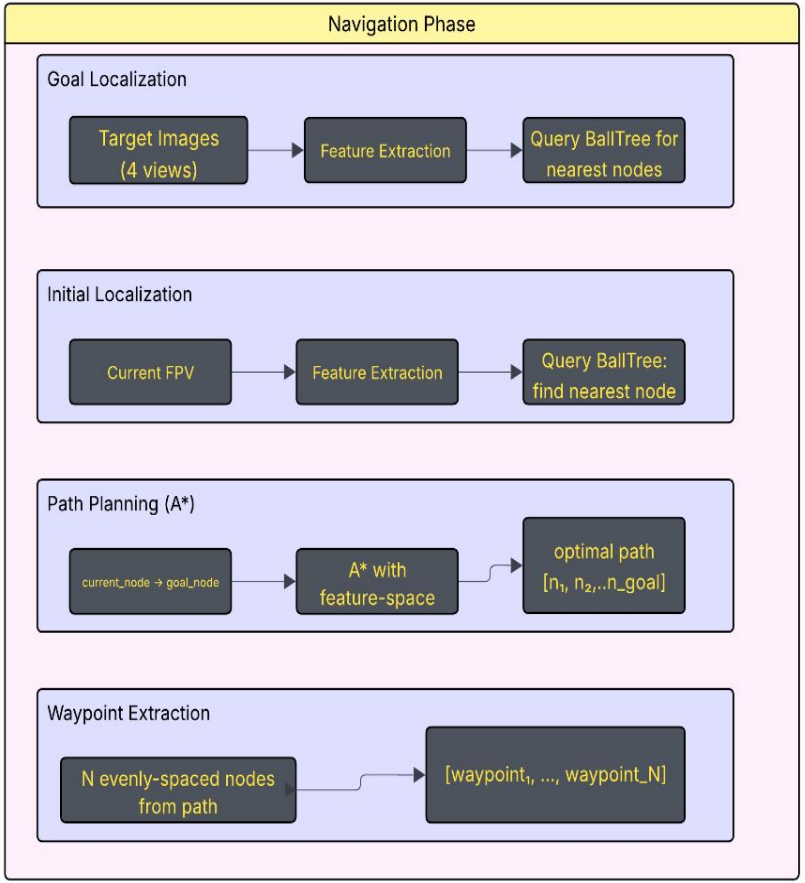
- we don't know the position and the map has not been provided.
- Must choose movements that lead closer to the goal while avoiding loops.
- We have to recognize different views of the same place.

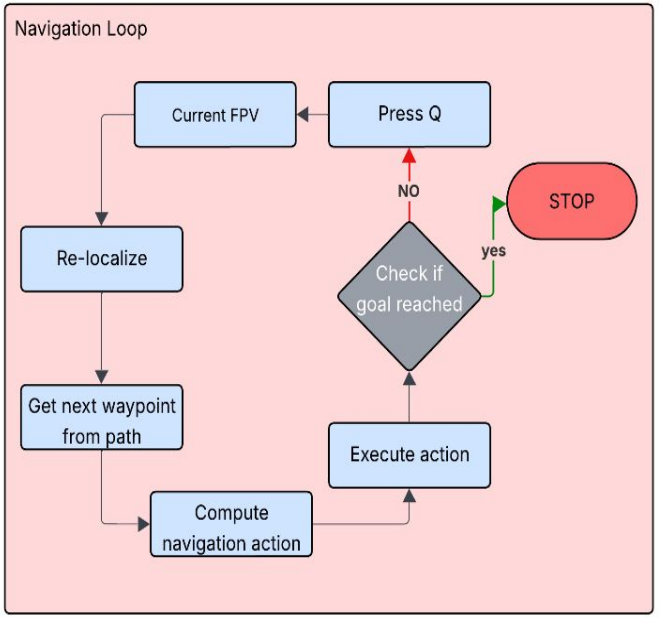
Core Strategy:

- Convert images into feature vectors using a CNN.
- Match the current view to the closest known location.
- Use a graph of connected viewpoints to plan the route.
- Convert planned path into simple actions (Left / Right / Forward)

Pipeline


Pre-Navigation
Exploration data [1575 images]
ResNet50 Feature Extractor
Feature matrix [1575 × 2048]
Graph Construction
Weighted graph G(V,E)
Index Building


Navigation Phase
Goal Localization
Target Images (4 views)
Feature Extraction
Query BallTree for nearest nodes
Initial Localization
Current FPV
Feature Extraction
Query BallTree: find nearest node
Path Planning (A*)
current_node → goal_node
A* with feature-space
optimal path [n1, n2,..n_goal]
Waypoint Extraction
N evenly-spaced nodes from path
[waypoint1, ..., waypoint_N]

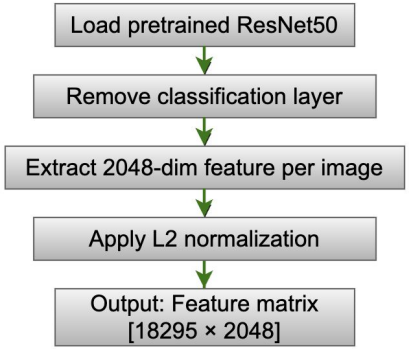

Navigation Loop
Current FPV
Press Q
NO
STOP
yes
Re-localize
Check if goal reached
Get next waypoint from path
Execute action
Compute navigation action

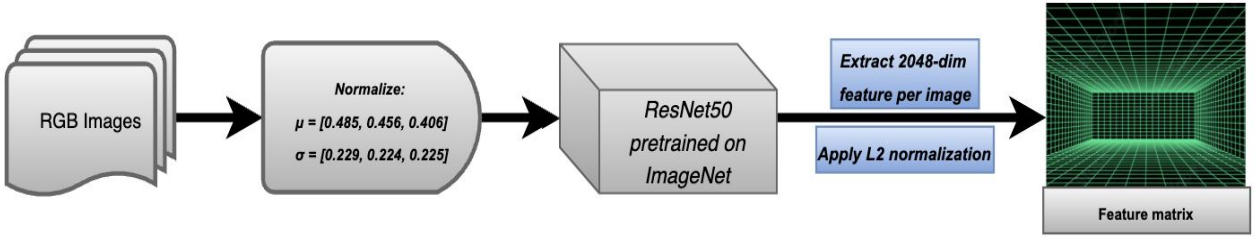
Deep Visual Feature Extraction

ResNet50 Layer Structure:

- **Input:** 224×224×3 (RGB images resized from 320×240)
- **Conv1 + MaxPool:** 7×7 convolution (64 filters) + 3×3 max pooling
- **16 Residual Blocks:** 4 stages with [3, 4, 6, 3] blocks per stage
- **Bottleneck Design:** Each block uses 1×1→3×3→1×1 convolutions
- **Global Average Pooling:** Reduces 7×7 spatial dimensions to 1×1
- **Output**: 2048-dimensional feature vector (before classification layer)

Feature extraction pipeline:





Features of ResNet50:

- **Adaptiveness:** ResNet-50 learns discriminative features from data; SIFT relies on DoG keypoints & gradients
- **Expressiveness:** Deep layers in ResNet-50 combine spatial and channel hierarchies leading to 2048-dimensional feature vector; SIFT uses a limited 128-dimensional descriptor per keypoint.
- **Density:** ResNet provides feature maps for *every pixel or patch*, enabling dense matching, whereas SIFT gives sparse keypoints which leads to limited coverage in textureless areas.

Graph Construction → Navigation Execution

Graph Builder

- Each image becomes a graph node
- Edges formed based on feature similarity (nearby-looking images)
- Two-Edge System:
  - **Temporal Edges**, **Spatial Edges**
- Index Building: Feature matrix F
  - Build Ball Tree with leaf_size=40
  - O(log n) nearest neighbor queries

Goal Detection

- System receives 4 target views of goal location
- Extract 2048-dim features from each target image.
- Find nearest matching nodes in exploration database
- Select best match using cosine similarity metric

Path Planning & Navigation:

- A* algorithm finds optimal path through visual graph
- **Heuristic:** h(node,goal) = 1.5 × $||\text{features}||_2$ (Euclidean distance)
- **Cost:** f(node) = cumulative_weights + heuristic_to_goal
- **Continuous localization:** match current FPV to path nodes
- Waypoint-guided navigation using FORWARD/LEFT/RIGHT actions

Challenges

- Insufficient exploration data results in lower training performance when using only 1,575 images from the midterm, whereas 18,000 images from the previous year yield better results.
- Sparse exploration leads to poor waypoint selection.
- Visual discontinuities occur between waypoints.
- Similar-looking corridor images confuse the ResNet model.

References

1. He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep residual learning for image recognition." CVPR. https://arxiv.org/abs/1512.03385
2. A* algorithm: Hart, P. E., et al. (1968). "A formal basis for the heuristic determination of minimum cost paths." IEEE Transactions on Systems Science and Cybernetics.
3. Baseline: https://github.com/ai4ce/vis_nav_player.git
4. Resnet50: https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet50.html
5. Torch: https://docs.pytorch.org/docs/stable/index.html
6. Scikit-Learn: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise_distances.html
7. NetworkX (Graph Building + A* Path Planning): https://networkx.org/documentation/stable/tutorial.html
8. OpenCV: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
9. Numpy: https://numpy.org/doc/1.25/user/index.html#user